

Import Libraries

```
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import SGD, RMSprop, Adam
from tensorflow.keras.callbacks import LearningRateScheduler
import matplotlib.pyplot as plt
```

Load Fashion MNIST Dataset

```
# Load Dataset
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Normalize Images
x_train = x_train / 255.0
x_test = x_test / 255.0

print(x_train.shape)
print(x_test.shape)
```

Output

```
(60000, 28, 28)
(10000, 28, 28)
```

Common Model Function

We'll use the same model for every optimizer.

```
def create_model():

    model = Sequential([
        Flatten(input_shape=(28,28)),

        Dense(128, activation='relu'),

        Dense(64, activation='relu'),

        Dense(10, activation='softmax')
    ])

    return model
```

Task 1: SGD Optimizer

```
model = create_model()

model.compile(
    optimizer=SGD(),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history_sgd = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test,y_test)
)

print("Final Training Accuracy:",
      history_sgd.history['accuracy'][-1])

print("Final Validation Accuracy:",
      history_sgd.history['val_accuracy'][-1])

print("Final Loss:",
      history_sgd.history['loss'][-1])
```

Typical Result

Training Accuracy $\approx$ 86%

Validation Accuracy $\approx$ 85%

Loss $\approx$ 0.40

Task 2: RMSProp Optimizer

```
model = create_model()

model.compile(

    optimizer=RMSprop(),

    loss='sparse_categorical_crossentropy',
```

```

        metrics=['accuracy']
    )

    history_rms = model.fit(
        x_train,
        y_train,
        epochs=5,
        validation_data=(x_test,y_test)
    )

    print("Validation Accuracy:",
          history_rms.history['val_accuracy'][-1])

```

Observation

Typical result

Validation Accuracy $\approx$ 88–89%

Compared to SGD:

- Faster convergence
- Lower loss
- Better validation accuracy

Task 3: Adam Optimizer (Learning Rate = 0.0005)

```

model = create_model()

optimizer = Adam(
    learning_rate=0.0005
)

model.compile(
    optimizer=optimizer,

```

```
loss='sparse_categorical_crossentropy',

metrics=['accuracy']

)

history_adam = model.fit(

    x_train,

    y_train,

    epochs=5,

    validation_data=(x_test,y_test)

)

print(history_adam.history['val_accuracy'][-1])
```

Typical Result

Validation Accuracy $\approx$ 89–91%

Observation

- Training becomes more stable.
- Loss decreases more quickly.
- Usually reaches higher accuracy in fewer epochs than SGD or RMSProp.

Task 4: Learning Rate Scheduler + Momentum SGD

Step 1: Define Scheduler

```
def scheduler(epoch, lr):

    if epoch > 0 and epoch % 2 == 0:

        return lr * 0.5

    return lr
```

Step 2: Callback

```
lr_callback = LearningRateScheduler(scheduler)
```

Step 3: Momentum Optimizer

```
optimizer = SGD(  
    learning_rate=0.01,  
    momentum=0.9  
)
```

Step 4: Train

```
model = create_model()  
  
model.compile(  
    optimizer=optimizer,  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
history = model.fit(  
    x_train,  
    y_train,  
    epochs=6,  
    validation_data=(x_test,y_test),  
    callbacks=[lr_callback]  
)
```

Learning Rate Changes

Epoch Learning Rate

1	0.0100
2	0.0100

Epoch Learning Rate

3 0.0050

4 0.0050

5 0.0025

6 0.0025